

Sistemet e Bazës së të Dhënave

Manipulimi i të Dhënave me SQL - Subqueries

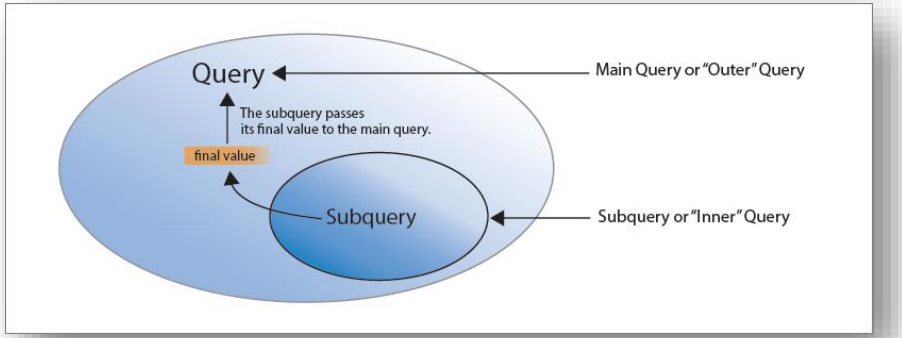
Elton Boshnjaku

Çfarë është një Subquery?

Një **subquery** është një **query (kërkesë) që ekzekutohet brenda një query tjetër** kryesore (parent query).

Subqueries shpesh **përdoren për të rikthyer një vlerë ose një grup të dhënash që përdoren si input për query-n kryesor**.

Ato **mund të vendosen në pjesë të ndryshme të një query**, si në **WHERE**, **FROM**, apo edhe **SELECT**.



Cili është qëllimi i një Subquery?

Thjeshton queries komplekse: Në vend që të krijohen query të gjata dhe të vështira për t'u lexuar, subqueries ndajnë logjikën në pjesë më të menaxhueshme.

Mundësojnë kushte dinamike: Subqueries ofrojnë fleksibilitet për të përpunuar dhe filtruar të dhënat bazuar në rezultate që nuk janë të disponueshme drejtpërdrejt.

Shembull i një subquery:

Query e cila i gjenë **studentët që kanë marrë nota mbi mesataren e klasës**.

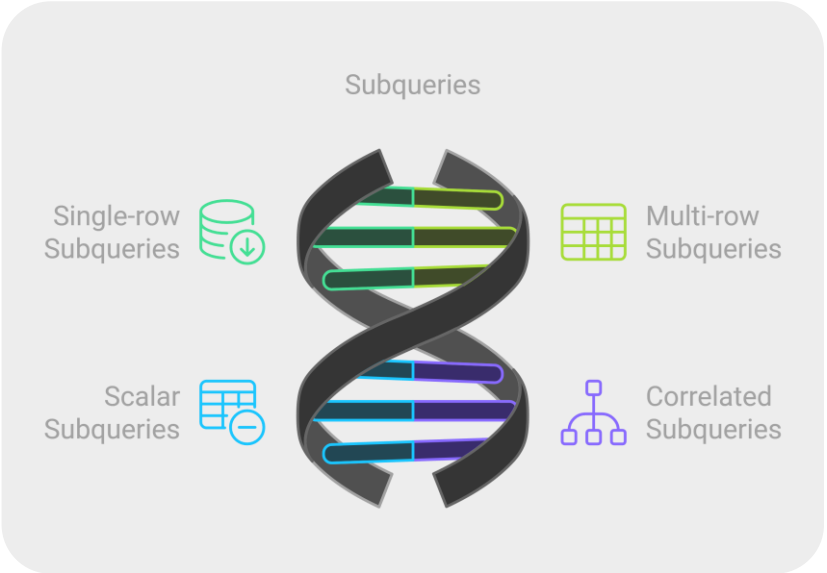
```
SELECT emri, nota
FROM studentet
WHERE nota > (SELECT AVG(nota) FROM studentet);
```

Llojet e Subqueries

Subqueries kategorizohen në disa lloje bazuar në mënyrën si rikthejnë rezultatet dhe si **ndërveprojnë me query-n kryesor**.

Llojet e Subqueries janë:

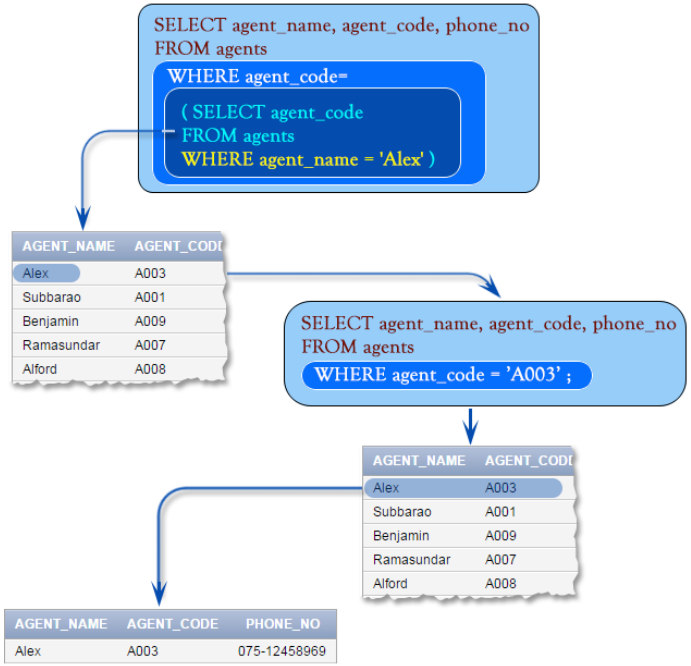
1. **Single-row Subqueries**
2. **Multi-row Subqueries**
3. **Scalar Subqueries**
4. **Correlated Subqueries**



Single-row Subqueries

Një **Single-row Subquery** është një **subquery që rikthen vetëm një rresht si rezultat**.

Këto subqueries janë **të dobishme kur nevojitet një vlerë specifike** për të plotësuar një kusht në query-n kryesor.



Karakteristikat Kryesore:

- Rikthen vetëm **një rresht**.
- Përdoret me **operatorë krahasues** si:
  - **=**
  - **>**
  - **<**
  - **<=**
  - **>=**
  - **!=**

Shembull Praktik - Single-row Subqueries

Gjej emrin e studentit me notën më të lartë.

```
SELECT emri
FROM studentet
WHERE nota = (SELECT MAX(nota) FROM studentet);
```

Shpjegim i kodit më lartë:

- Subquery (SELECT MAX(nota) FROM studentet) rikthen notën më të lartë.
- Query kryesor e përdor atë rezultat për të gjetur studentin përkatës.

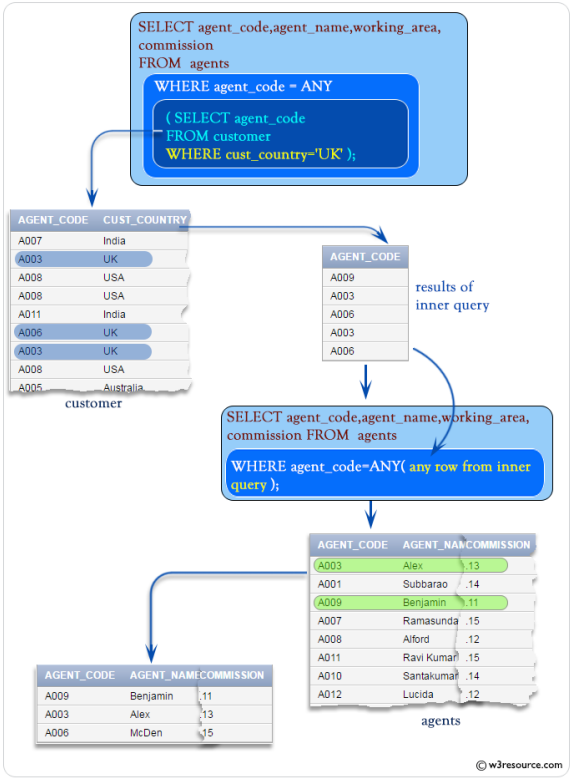
Nëse subquery rikthen më shumë se një rresht, SQL do të raportojë një gabim. Kjo është arsyeja pse është e rëndësishme që subquery të jetë i dizajnuar për të rikthyer vetëm një rezultat unik.

Multi-row Subqueries

Multi-row Subqueries janë **subqueries që rikthejnë më shumë se një rresht si rezultat**, shpesh duke u përdorur për të filtruar të dhëna nga query kryesor në bazë të një grupi vlerash.

Karakteristikat Kryesore:

- Rikthejnë **shumë rreshta**, zakonisht me një kolonë.
- Kombinohen me **operatorë** si:
  - **IN**
  - **ANY**
  - **ALL**
  - **EXISTS**.



Shembull Praktik – Multi-row Subqueries

Gjej emrat e studentëve që kanë marrë nota në lëndën "Matematikë":

```
SELECT emri
FROM studentet
WHERE nota IN (SELECT nota FROM provimet WHERE lenda = 'Matematike');
```

Shpjegim i kodit më lartë:

- Subquery (SELECT nota FROM provimet WHERE lenda = 'Matematike') rikthen të gjitha notat për lëndën "Matematikë".
- Query kryesor përdor këto vlera për të krahasuar dhe filtruar të dhënat e studentëve.

*Nëse përdorni operatorë si **=** për një multi-row subquery, do të shkaktohet një error. Duhet të siguroheni që operatorët të jenë të përshtatshëm për shumë rreshta, si **IN, ANY, ALL ose EXISTS**.*

Scalar Subqueries

Një **Scalar Subquery** është një **subquery që rikthen një vlerë të vetme** (scalar value) si rezultat. Kjo vlerë **mund të përdoret në çdo pjesë të query-t ku pritet një vlerë individuale**, si në një kolonë të re apo në kushte krahasimi.

Karakteristikat Kryesore:

- Rikthen **vetëm një vlerë**, jo më shumë se një rresht ose një kolonë.
- Mund të përdoret në :
    - **SELECT** për të shtuar kolona të reja.
    - **WHERE** për kushte dinamike.

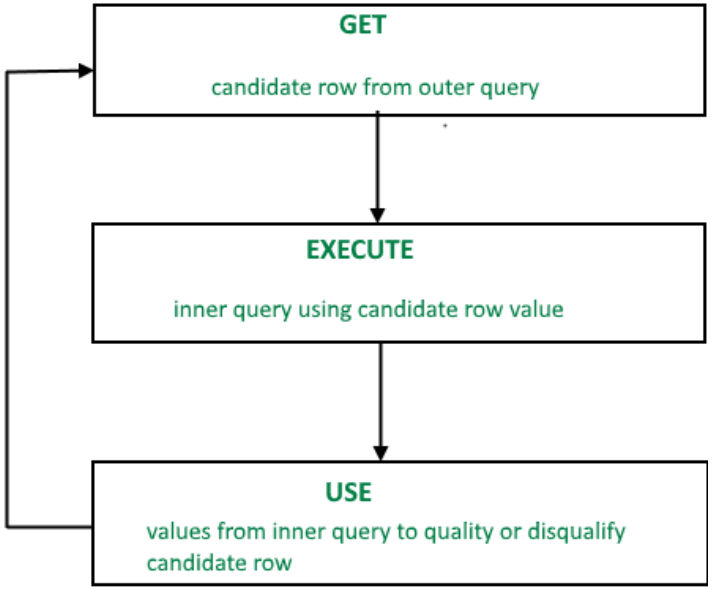
Shembull:

```
SELECT emri,
       (SELECT COUNT(*) FROM provimet WHERE provimet.student_id = studentet.student_id) AS numri_provimeve
FROM studentet;
```

Correlated Subqueries

Correlated Subqueries janë **subqueries që referojnë query-n e jashtëm (outer query) dhe ekzekutohen për secilin rresht të tij**.

Ato **varen nga të dhënat e rreshtit aktual** të query-t kryesor dhe **nuk mund të ekzekutohen të pavarura**.



Karakteristikat Kryesore:

- **Dinamik:** Ekzekutohet një herë për çdo rresht të query-t kryesor.
- **Varësi:** Referon kolonat nga query i jashtëm.
- **Performanca:** Mund të jetë më e ngadaltë për shkak të ekzekutimit të shumëfishtë.

Shembull Praktik – Correlated Subqueries

Gjej emrat e studentëve që kanë marrë të paktën një notë mbi 8:

```
SELECT emri
FROM studentet s
WHERE EXISTS (
    SELECT 1
    FROM provimet p
    WHERE p.student_id = s.student_id
      AND p.nota > 8
);
```

Shpjegim i kodit më lartë:

- Subquery (SELECT 1 FROM provimet WHERE p.student_id = s.student_id AND p.nota > 8) ekzekutohet për secilin student nga query kryesor.
- Kontrollon nëse ekziston një notë mbi 8 për studentin aktual.

Operatorët e Përdorur në Subqueries

IN - Kontrollon nëse një vlerë përputhet me cilëndo vlerë të rikthyer nga subquery.

NOT IN - Kontrollon nëse një vlerë nuk ndodhet në grupin e vlerave të subquery.

EXISTS - Kontrollon nëse subquery rikthen të paktën një rresht.

NOT EXISTS - Kontrollon nëse subquery nuk rikthen asnjë rresht.

ANY - Kontrollon nëse një vlerë përputhet me ndonjë vlerë të rikthyer nga subquery.

ALL - Kontrollon nëse një vlerë përputhet me të gjitha vlerat e rikthyera nga subquery.

=, <, >, <=, >=, != - Përdoren për krahasime të drejtpërdrejta me një subquery që rikthen një vlerë të vetme.

Subqueries në FROM

Subqueries në **FROM** përdoren për të krijuar **tabela të përkohshme** (tabela të derivuara) që query kryesor **mund t'i trajtojë si burime të dhënash**.

Karakteristikat Kryesore:

- Vepron si një burim i përkohshëm i të dhënave brenda query kryesor.
- E dobishme për organizimin dhe thjeshtimin e query-ve komplekse.

Avantazhet:

- **Thjeshtim:** Thjeshton query-n kryesor duke ndarë logjikën në hapa të qartë.
- **Fleksibilitet:** Mund të përdoret për t'i trajtuar subqueries si tabela virtuale.

Shembull Praktik – Subqueries në FROM

Gjej mesataren e numrit të studentëve në secilën lëndë:

```
SELECT lenda, AVG(numri_studentet) AS mesatarja
FROM (
    SELECT lenda, COUNT(student_id) AS numri_studentet
    FROM provimet
    GROUP BY lenda
) AS tabela_derivuar
GROUP BY lenda;
```

Shpjegim i kodit më lartë:

- Subquery në FROM krijon një tabelë të përkohshme *tabela_derivuar*, ku numërohet numri i studentëve për çdo lëndë.
- Query kryesor llogarit mesataren e numrit të studentëve për secilën lëndë.

Subqueries në SELECT

Subqueries në **SELECT përdoren për të krijuar kolona të reja** duke **llogaritur** ose **rikthyer vlera specifike për secilin rresht të query kryesor**.

Karakteristikat Kryesore:

- Rikthejnë një vlerë të vetme për secilin rresht të query kryesor.
- Llogarisin vlera dinamike që përfshihen si kolona të reja në rezultatin përfundimtar.
- Nuk ndikon në filtrimin apo grupimin e të dhënave në query kryesor.

Avantazhet:

- **Dinamike:** Gjeneron rezultate unike për secilin rresht.
- **Efiçente:** Kursen nevojën për query shtesë, duke integruar subquery në rezultatin përfundimtar.

Subquery duhet të rikthejë vetëm një vlerë për secilin rresht, përndryshe SQL raporton error.

Shembull Praktik – Subqueries në SELECT

Gjej emrin e studentëve dhe mesataren e notave të tyre:

```
SELECT emri,
       (SELECT AVG(nota)
        FROM provimet
        WHERE provimet.student_id = studentet.student_id) AS mesatarja
FROM studentet;
```

Shpjegim i kodit më lartë:

- Subquery (SELECT AVG(nota) FROM provimet WHERE provimet.student_id = studentet.student_id) **llogarit mesataren e notave** për secilin student.
- Rezultati i subquery shfaqet si një kolonë e re e quajtur mesatarja.

Konsiderata për Performancën

Subqueries, megjithëse të fuqishme dhe fleksibile, **mund të kenë një ndikim negativ në performancë kur përdoren në dataset-e të mëdha** ose në **skenarë të ndërlikuar**.

Konsiderata Kryesore për Performancën:

Shpejtësia e Subqueries:

Subqueries mund të jenë më të ngadalta krahasuar me JOIN, sidomos nëse ekzekutohen në mënyrë të përsëritur (p.sh., Correlated Subqueries).

Përdorimi i Indekseve:

Sigurohuni që kolonat e përdorura në subqueries dhe query-n kryesor të kenë indekse për të përmirësuar kohën e kërkimit.

Transformimi në JOIN:

Në shumë raste, një subquery mund të transformohet në një JOIN, që shpesh është më efikas.

Shembull i Përformancës së Optimizuar

Subquery: Gjej emrat e studentëve që kanë dhënë të paktën një provim me notë mbi 8:

```
SELECT emri
FROM studentet s
WHERE EXISTS (
    SELECT 1
    FROM provimet p
    WHERE p.student_id = s.student_id
      AND p.nota > 8
);
```

Optimizuar me JOIN:

```
SELECT DISTINCT s.emri
FROM studentet s
JOIN provimet p ON s.student_id = p.student_id
WHERE p.nota > 8;
```

Subqueries vs. JOINs

Subqueries:

- Preferohen për lexueshmëri kur analizat janë të ndara në hapa logjikë.
- Janë më të thjeshta për raste me kërkesa specifike ose kur rezultati i një subquery përdoret direkt.

Joins:

- Tipikisht më të shpejta për dataset-e të mëdha, pasi përdorin algoritme të optimizuara për bashkime të rreshtave (JOIN).
- Janë më të mira kur të dhënat nga tabela të ndryshme kombinohen për analizë të integruar.

Përdorimet e Subqueries:	Përdorimet e JOINs:
Kur rezultati i një query përdoret si input për një tjetër.	Për të bashkuar të dhënat nga shumë tabela në një query.
Për kushte të dinamike në seksionin WHERE.	Për dataset-e të mëdha që kërkojnë performancë optimale.
Kur është e nevojshme të rikthehet një vlerë e vetme ose një grup vlerash.	Për të shmangur ekzekutime të përsëritura në correlated subqueries.

Subqueries vs. JOINs

	Join	Subquery
Type	Joins together rows from different tables based on a **common column.**	Executes a **separate query** inside the main query.
Purpose	Combines **data from multiple tables** into a single result set.	Filters, sorts, or aggregates **data within a single table** or query.
Syntax	Uses the **JOIN keyword** and specifies the join type and join condition.	Written **within parentheses** after a WHERE or HAVING clause.
Performance	Can be **more efficient for joining large tables** or when many rows match the join condition.	Can be slower and **more resource-intensive for large datasets** or complex queries.
Flexibility	Allows **complex queries with multiple join conditions** and nested joins.	Offers **more flexibility for filtering and sorting data,** and may be used in more complex query structures.
Clarity	Can **make queries easier to read and understand,** especially for simple joins	This can **make queries more difficult to read and understand,** especially for nested or complex subqueries
Duplicates	Can produce **duplicate rows** if there are multiple matching rows in the joined tables	May be **used to remove duplicates from a query** result or to identify duplicate values in a single table



Do you have any questions?

elton.boshnjaku@ubt-uni.net
www.ubt-uni.net

Referencat

Fundamentals of Database Systems
Authors: Ramez Elmasri and Shamkant B. Navathe
Edition: 7th Edition
Year: 2016

Practical SQL: A Beginner's Guide to Storytelling with Data
Author: Anthony DeBarros
Edition: 1st Edition
Year: 2018

SQL All-in-One For Dummies
Author: Allen G. Taylor
Edition: 3rd Edition
Year: 2019